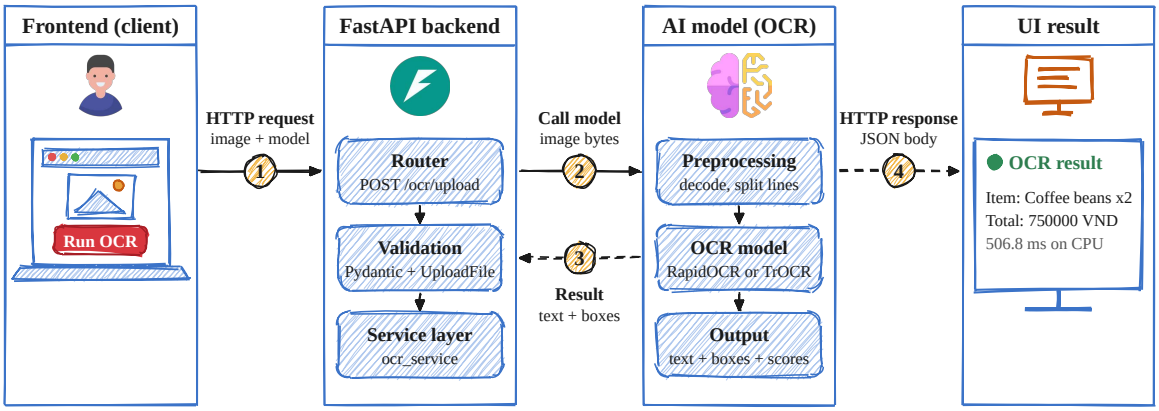
From Notebook to Production
Serving ML Models with FastAPI

Nguyễn Đăng Nhã **Đinh Quang Vinh**

Content Revision: **Trần Quang Minh**

I. Motivation

Hãy hình dung một kịch bản quen thuộc trong các dự án AI. Sau nhiều tuần thu thập dữ liệu và huấn luyện, chúng ta có trong tay một model nhận diện chữ trên hóa đơn với độ chính xác đáng tin cậy. Model ấy hoạt động tốt, nhưng chỉ đang hoạt động bên trong một notebook hoặc một script Python trên máy của người huấn luyện. Bộ phận kế toán muốn tải ảnh hóa đơn lên và nhận về văn bản; team phát triển ứng dụng mobile muốn tích hợp tính năng quét hóa đơn; một hệ thống khác trong công ty muốn gọi model theo batch mỗi đêm. Không ai trong số họ có thể tải mode, mở notebook và chạy từng cell code để lấy kết quả. Hình 1 minh họa đích đến mà tài liệu này hướng tới; đó là hướng tới xây dựng một pipeline sử dụng fastAPI thực hiện xây dựng hệ thống serving model nhận request từ người dùng.



Hình 1: Pipeline triển khai mô hình OCR dùng FastAPI. Người dùng chọn ảnh hóa đơn, model và device trên giao diện; frontend gửi request đến FastAPI backend, nơi request được kiểm tra rồi chuyển cho OCR model đã được load sẵn, và văn bản nhận dạng được trả về để hiển thị cho người dùng.

Vấn đề nằm ở chỗ những hệ thống cần sử dụng mô hình thường không chạy trong cùng một tiến trình, thậm chí có thể được viết bằng những ngôn ngữ lập trình khác nhau và chạy trên những máy tính khác nhau. Chúng cần một cách giao tiếp chung để gửi dữ liệu đến mô hình và nhận

kết quả trả về. Trong các ứng dụng web, HTTP là một giao thức phổ biến cho mục đích này, còn API cung cấp một giao diện để các hệ thống khác sử dụng chức năng của mô hình mà không cần biết cách mô hình được triển khai bên trong. Khi được đưa vào một dịch vụ có khả năng tiếp nhận request và trả về kết quả qua API, mô hình đang được triển khai theo cách thường gọi là **model serving**.

Tuy nhiên, đưa một mô hình lên API không chỉ đơn giản là nhận dữ liệu, gọi hàm dự đoán rồi trả kết quả. Nếu tự xây dựng toàn bộ phần giao tiếp, chúng ta phải xử lý nhiều công việc bên cạnh logic của mô hình: đọc và phân tích request, kiểm tra kiểu dữ liệu, phát hiện các trường bị thiếu hoặc có giá trị không hợp lệ, cũng như trả về thông báo lỗi để phía client biết vấn đề nằm ở đâu. Tài liệu API cũng cần được duy trì song song với mã nguồn để tránh tình trạng tài liệu không còn khớp với cách hệ thống thực sự hoạt động. Ngoài ra, server phải có khả năng tiếp nhận nhiều request, quản lý việc sử dụng tài nguyên và nạp mô hình đúng thời điểm. Chẳng hạn, nếu mô hình được nạp lại từ đĩa trong mỗi request, thời gian xử lý sẽ tăng lên đáng kể và tài nguyên bị sử dụng không cần thiết. Đây đều là những vấn đề thuộc về quá trình triển khai, chứ không phải bản thân bài toán AI.

FastAPI [1] giúp giải quyết nhiều vấn đề trong số đó bằng cách tận dụng các cơ chế sẵn có của Python. Thay vì tự viết các đoạn mã kiểm tra dữ liệu cho từng request, chúng ta có thể khai báo kiểu dữ liệu bằng **type hints** và sử dụng Pydantic [2] để xác thực dữ liệu đầu vào. FastAPI dựa trên các khai báo này để phân tích request, thực hiện chuyển đổi kiểu dữ liệu phù hợp và trả về lỗi có cấu trúc, thường với mã HTTP 422, khi dữ liệu không đáp ứng yêu cầu. Các khai báo cũng được dùng để tự động tạo đặc tả OpenAPI và trang tài liệu tương tác tại `/docs`, giúp người phát triển kiểm tra và thử nghiệm API ngay trên trình duyệt.

Ở phía máy chủ, FastAPI được xây dựng trên ASGI và hỗ trợ `async`/`await`, cho phép ứng dụng xử lý các hoạt động I/O bất đồng bộ mà không phải chờ từng thao tác hoàn tất theo cách tuần tự. Một ASGI server như Uvicorn [3] chịu trách nhiệm chạy ứng dụng và tiếp nhận các kết nối mạng. Bên cạnh đó, FastAPI cung cấp các cơ chế như *lifespan* để quản lý tài nguyên trong suốt vòng đời ứng dụng, *dependency injection* để tổ chức và cung cấp các thành phần dùng chung, cùng *router* để chia API thành những phần có cấu trúc rõ ràng. Nhờ vậy, người phát triển có thể dành nhiều thời gian hơn cho logic của ứng dụng AI thay vì phải tự xây dựng mọi thành phần phục vụ giao tiếp và vận hành.

Tài liệu này tiếp cận FastAPI thông qua quá trình xây dựng một hệ thống hoàn chỉnh, thay vì lần lượt giới thiệu từng tính năng riêng lẻ. Mỗi bước bắt đầu từ một phiên bản đơn giản, sau đó giải quyết những vấn đề nảy sinh khi hệ thống được mở rộng.

Toàn bộ mã nguồn mọi người có thể xem tại repository `dangnha/routes-for-AI-inference` [4]. Mỗi bước được tổ chức trong một thư mục riêng, giúp người đọc dễ dàng theo dõi sự thay đổi của hệ thống. Các kết quả minh họa trong tài liệu được lấy từ quá trình chạy thực tế những project tương ứng. Bảng dưới đây tổng hợp các thuật ngữ sẽ được sử dụng xuyên suốt tài liệu.

Bảng Giải Thích Thuật Ngữ

Thuật Ngữ	Giải thích
API	Giao diện cho phép chương trình khác gọi chức năng của hệ thống qua mạng, ở đây là qua HTTP.
Endpoint (route)	Cặp HTTP method và URL path được gắn với một hàm Python, ví dụ `POST /predict`.
Inference	Dùng model đã huấn luyện để dự đoán trên dữ liệu mới; khác với training, bước fit model để tạo artifact.
Pydantic schema	Class kế thừa `BaseModel` mô tả tên, kiểu và ràng buộc của dữ liệu.
Lifespan	Hàm chạy khi server startup và shutdown, dùng để nạp và giải phóng model đúng một lần.
Service layer	Lớp chứa logic nghiệp vụ và điều phối inference, tách biệt khỏi phần xử lý HTTP.
Dependency injection	Cơ chế `Depends()` cho phép FastAPI truyền object (ví dụ service) vào endpoint.
Response contract	Cấu trúc response, kể cả lỗi, mà server cam kết trả về, khai báo bằng `response_model`.
Model registry	Thành phần quản lý vòng đời model: đăng ký, lazy load và unload theo device.

Mục lục

I. Motivation **1**

II. Thiệu thiệu về HTTP, API và vai trò của FastAPI **5**

III. Triển khai fastAPI cho Python Function **6**

III.1. Nhận dữ liệu qua path, query và request body 8

IV. Serve một pretrained model **9**

IV.1. Vấn đề của việc load model trong endpoint 10

IV.2. Load model một lần với lifespan và tách service layer 11

V. Kiểm tra dữ liệu đầu vào với Pydantic **13**

VI. Tổ chức source code theo layer **15**

VII. Thiết kế response contract ổn định **17**

VII.1. Dùng response_model để kiểm soát output 17

VII.2. Chuẩn hóa lỗi bằng domain exception và status code 19

VIII. Multi-model AI server cho bài toán OCR **21**

VIII.1. Routers cho từng tác vụ và model registry 22

VIII.2. Nhận file ảnh với UploadFile 23

VIII.3. async def và inference có tính blocking 24

VIII.4. Error có nghĩa và CORS 25

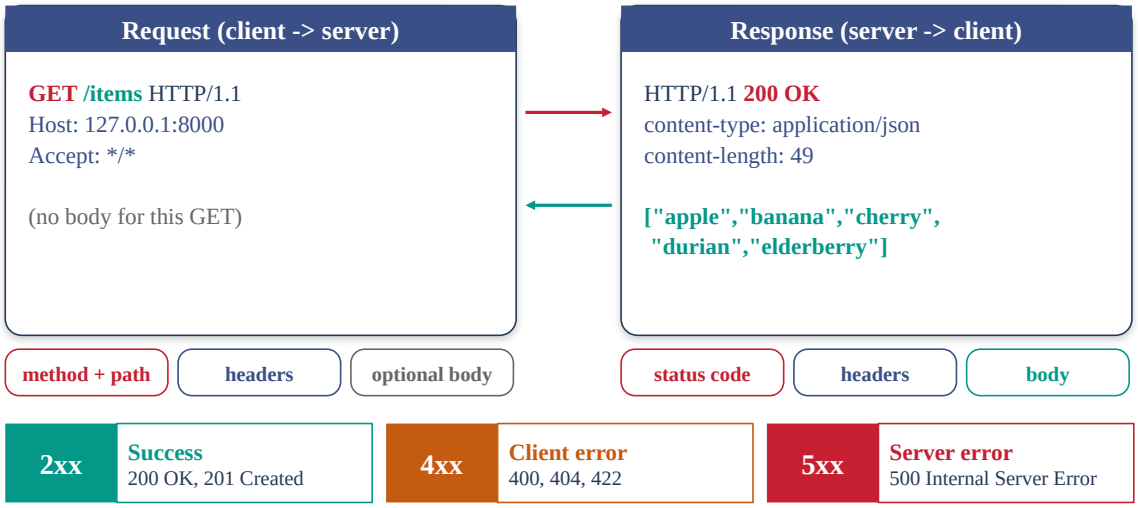
VIII.5. Chạy toàn bộ hệ thống 26

IX. Tổng kết **27**

Phụ lục **29**

II. Thiệu thiệu về HTTP, API và vai trò của FastAPI

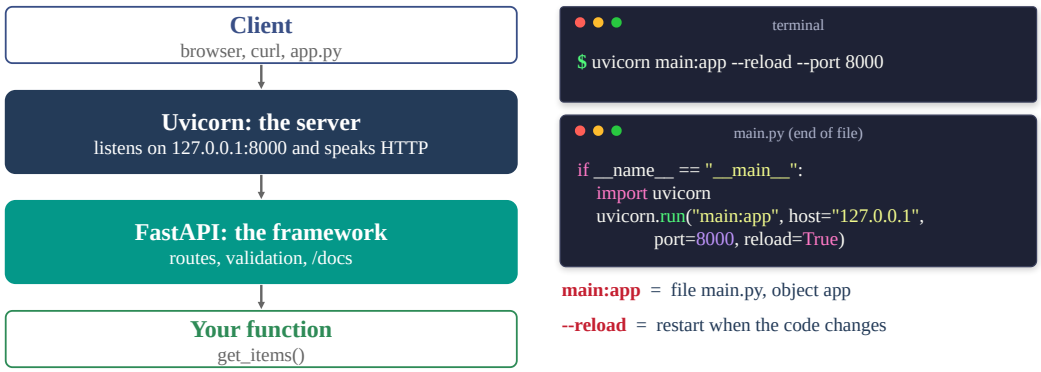
Trước khi viết dòng code đầu tiên, chúng ta cần thống nhất cách hai chương trình nói chuyện với nhau. Một chương trình khi chạy trở thành một process; hai process, có thể nằm trên hai máy khác nhau, trao đổi dữ liệu qua mạng bằng một giao thức chung. Với web API, giao thức đó là HTTP. Client gửi một request gồm method, path, headers và body tùy chọn; server trả về một response gồm status code, headers và body tùy chọn; khi có body, với web API body thường ở định dạng JSON, còn một số response như `204 No Content` không có body. Tập hợp các endpoint cùng định dạng request và response của chúng tạo thành **API contract** giữa frontend và backend, như được minh họa trong Hình 2.



Hình 2: Một lượt trao đổi HTTP điển hình. Request `GET /items` mang method, path và headers; còn response mang status code, headers và body JSON; status code cho client biết ngay kết quả thuộc nhóm thành công (2xx), lỗi phía client (4xx) hay lỗi phía server (5xx).

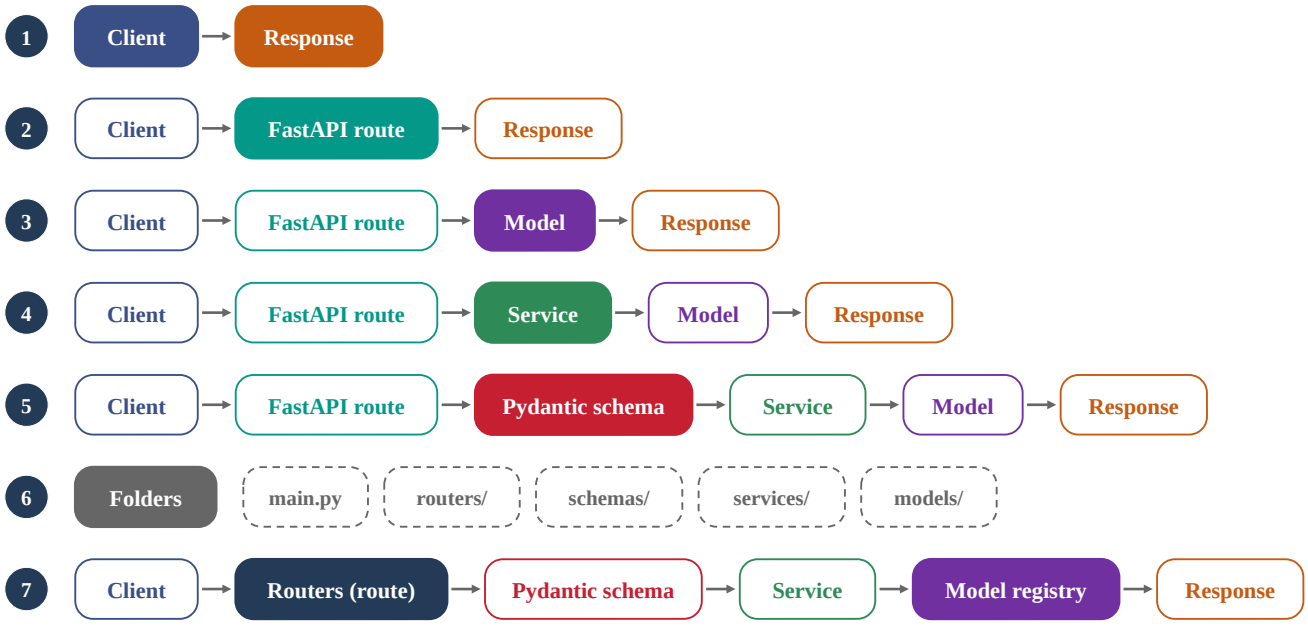
Hai method được dùng nhiều nhất trong tài liệu là GET, dùng để đọc dữ liệu và thường không có body, và POST, dùng để gửi dữ liệu như đặc trưng của một bông hoa hoặc một file ảnh. Status code giúp client hiểu kết quả mà không cần phân tích body. `200 OK` cho thành công, `404 Not Found` khi tài nguyên không tồn tại, `422 Unprocessable Entity` khi dữ liệu gửi lên không qua được bước validation, gồm sai kiểu dữ liệu, thiếu trường bắt buộc hoặc vi phạm ràng buộc như giá trị nằm ngoài khoảng cho phép, và `500 Internal Server Error` khi server gặp lỗi bất ngờ.

Một điểm thường gây nhầm lẫn là sự phân vai giữa FastAPI và Uvicorn. Uvicorn [3] là một **ASGI server**, thực hiện các chức năng như mở cổng mạng, chấp nhận kết nối và xử lý giao thức HTTP ở mức thấp. FastAPI là một **framework** cho phép nhận request đã được Uvicorn phân tích, chọn đúng route, kiểm tra dữ liệu, gọi hàm Python và đóng gói kết quả thành response. Vì vậy, lệnh `uvicorn main:app --reload` có nghĩa là khởi động Uvicorn với object `app` được định nghĩa trong file `main.py` và tự động nạp lại khi code thay đổi (Hình 3).



Hình 3: Sự phân vai giữa hai thành phần khi chạy `uvicorn main:app`. Uvicorn lắng nghe trên cổng mạng và xử lý giao thức HTTP, còn FastAPI định tuyến request đến đúng hàm Python và tạo response.

Với nền tảng này, ta có thể bắt đầu xây dựng hệ thống. Hình 4 mô tả toàn bộ lộ trình của tài liệu. Kiến trúc lớn dần qua từng bước, và mỗi bước chỉ bổ sung đúng thành phần cần thiết để khắc phục giới hạn của bước trước.



Hình 4: Cách thiết kế nội dung của tài liệu. Từ một cặp client và response đơn giản, kiến trúc lần lượt được bổ sung FastAPI route, model, service layer, Pydantic schema, cấu trúc thư mục theo layer, và cuối cùng là routers kết hợp model registry.

III. Triển khai fastAPI cho Python Function

Bước đầu tiên là trường hợp đơn giản nhất. Giả sử ta có hàm `get_items()` trả về một danh sách. Hàm này hoạt động hoàn hảo khi được gọi trong cùng chương trình, nhưng một người dùng khác, một trang web hay một ứng dụng viết bằng ngôn ngữ khác hoàn toàn không có cách nào chạm vào process của ta để gọi nó. Thứ còn thiếu là một cầu nối giữa thế giới HTTP và

hàm Python, và trong FastAPI cầu nối đó chính là **endpoint**. Code của bước này nằm trong thư mục `01-simple-api` của repo [4]; đoạn code dưới đây có ba phần gồm hàm Python thuần, object `app`, và các decorator gắn hàm vào URL.

01-simple-api/main.py: biến hàm Python thành endpoint

```python
from fastapi import FastAPI

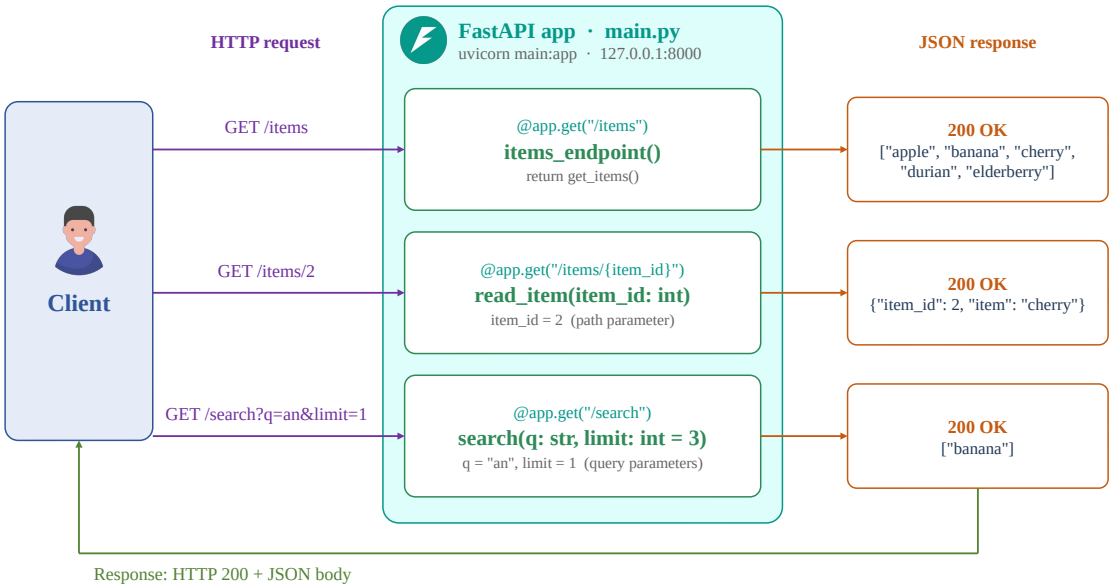
def get_items() -> list[str]:
    return ["apple", "banana", "cherry", "durian", "elderberry"]

app = FastAPI(title="Simple API - List", version="1.0.0")

@app.get("/items", summary="Return a list of items")
def items_endpoint() -> list[str]:
    return get_items()

@app.get("/health", summary="Liveness check")
def health():
    return {"status": "ok"}
```

Decorator `@app.get("/items")` đăng ký cặp method GET và path `/items` với hàm `items_endpoint`; khi request đến, FastAPI gọi hàm và tự chuyển giá trị `list[str]` thành một JSON array. Đáng chú ý là `get_items()` vẫn là Python thuần và không biết gì về HTTP. Nguyên tắc tách logic khỏi lớp giao tiếp này sẽ được giữ xuyên suốt tài liệu. Để hiểu hơn về ví dụ này, hãy thực hiện chạy `uvicorn main:app --reload --port 8000` rồi mở `http://127.0.0.1:8000/docs`; Swagger UI cho phép gửi request trực tiếp từ trình duyệt. Hình 5 đặt endpoint này vào bức tranh chung.



Hình 5: Ba endpoint của `01-simple-api` và phần mở rộng: `GET /items` gọi `items_endpoint()` và trả về toàn bộ danh sách; `GET /items/2` truyền path parameter `item_id = 2` cho `read_item` và nhận `{"item_id": 2, "item": "cherry"}`; `GET /search?q=an&limit=1` truyền hai query parameter cho `search` và nhận `["banana"]`. Mọi kết quả đều được FastAPI chuyển thành JSON response với status 200.

III.1. Nhận dữ liệu qua path, query và request body

Một endpoint chỉ trả dữ liệu cố định thì ít giá trị; trong thực tế, client cần gửi dữ liệu lên. FastAPI hỗ trợ ba cách: path parameter nằm ngay trong path như `/items/2`, query parameter nằm sau dấu `?` như `/search?q=an&limit=1`, và request body là JSON gửi kèm request POST. Ví dụ mở rộng sau minh họa cả ba cách.

Mở rộng 01-simple-api: path, query và JSON body

```python
ITEMS = get_items()

@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id, "item": ITEMS[item_id]}

@app.get("/search")
def search(q: str, limit: int = 3):
    return [name for name in ITEMS if q in name][:limit]

class NewItem(BaseModel):
    name: str

@app.post("/items")
def add_item(item: NewItem):
    ITEMS.append(item.name)
    return {"added": item.name, "count": len(ITEMS)}
```

Trong ví dụ này, function signature quyết định nguồn dữ liệu: `item_id` xuất hiện trong path template nên là path parameter; trong `search`, `q` và `limit` có kiểu đơn (`str`, `int`) và không nằm trong path nên được đọc từ query string; `q` không có giá trị mặc định nên là bắt buộc (thiếu `q` sẽ nhận 422 `missing`), còn `limit` có mặc định 3 nên là tùy chọn; còn `item` có kiểu `BaseModel` nên được đọc từ JSON body. Nói cách khác, kiểu và vị trí khai báo quyết định tham số được lấy từ đâu, còn giá trị mặc định chỉ quyết định tham số có bắt buộc hay không. Khai báo kiểu còn kéo theo kiểm tra kiểu. Nếu client gọi `GET /items/abc`, giá trị `"abc"` không thể chuyển sang `int`, nên FastAPI trả về 422 và hàm `read_item` không hề được gọi:

Response của GET /items/abc

```
HTTP 422 {"detail": [{"type": "int_parsing", "loc": ["path", "item_id"],
  "msg": "Input should be a valid integer, unable to parse string as an integer", "
                                          input": "abc"}]}
```

Ngoài Swagger UI, có thể thử API bằng `curl http://127.0.0.1:8000/items` hoặc bằng thư viện `requests` trong Python. Cần lưu ý rằng địa chỉ `127.0.0.1` chỉ cho phép chính máy đang chạy server truy cập; để máy khác gọi được, server phải lắng nghe trên `0.0.0.0` hoặc được triển khai lên một máy có địa chỉ công khai.

IV. Serve một pretrained model

Hàm `get_items()` ở bước trước chỉ là một phép tính tức thời. Khi thay nó bằng một AI model, bài toán thay đổi về bản chất. Model phải được huấn luyện trước, lưu thành file và nạp vào bộ nhớ trước khi dự đoán, và việc nạp này có chi phí. Ví dụ xuyên suốt của bước này là Iris classification: nhận bốn đặc trưng của một bông hoa (sepal length, sepal width, petal length, petal width) và dự đoán loài. Trước hết cần phân biệt hai giai đoạn. **Training** đọc dữ liệu và fit model; trong ví dụ này, training được chạy trước khi serving để tạo artifact, và có thể huấn luyện lại khi cập nhật model. **Inference** chạy bên trong server cho từng request và cần nhẹ, nhanh. Script training trong thư mục `02-serve-model` tạo ra artifact như sau:

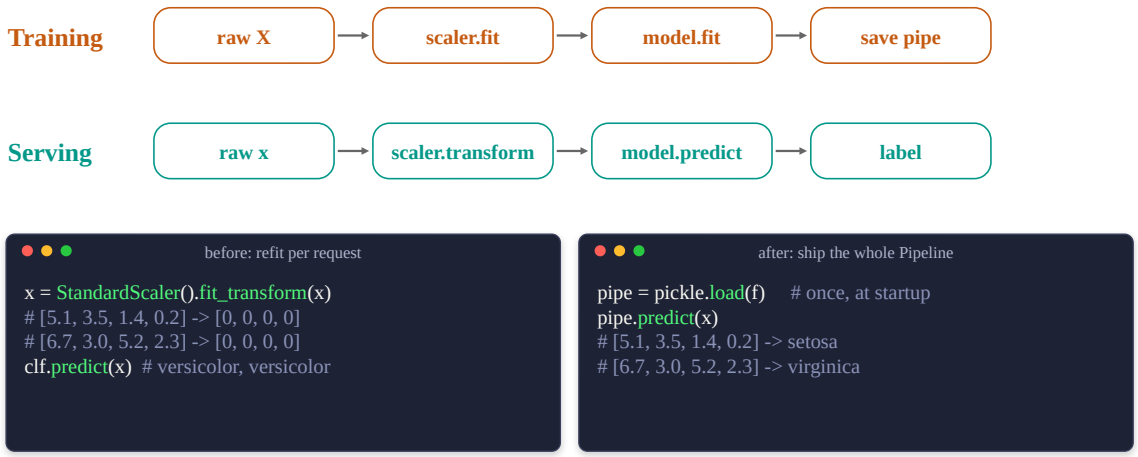
02-serve-model/train_model.py: train một lần, lưu model.pkl

```python
iris = load_iris()
model = LogisticRegression(max_iter=200)
model.fit(iris.data, iris.target)

with open("model.pkl", "wb") as f:
    pickle.dump(model, f)
# Lưu thêm metadata: tên feature và tên nhãn để lúc inference hiển thị cho đúng.
with open("model_meta.json", "w", encoding="utf-8") as f:
    json.dump({"feature_names": list(iris.feature_names),
               "target_names": list(iris.target_names)}, f)
```

Bên cạnh `model.pkl`, script lưu thêm `model_meta.json` chứa tên nhãn, vì model chỉ trả về chỉ số lớp (0, 1, 2) trong khi API nên trả về nhãn dễ đọc như `setosa`. Cần lưu ý rằng `pickle.load` có thể thực thi code tùy ý, nên chỉ nạp artifact từ nguồn đáng tin cậy.

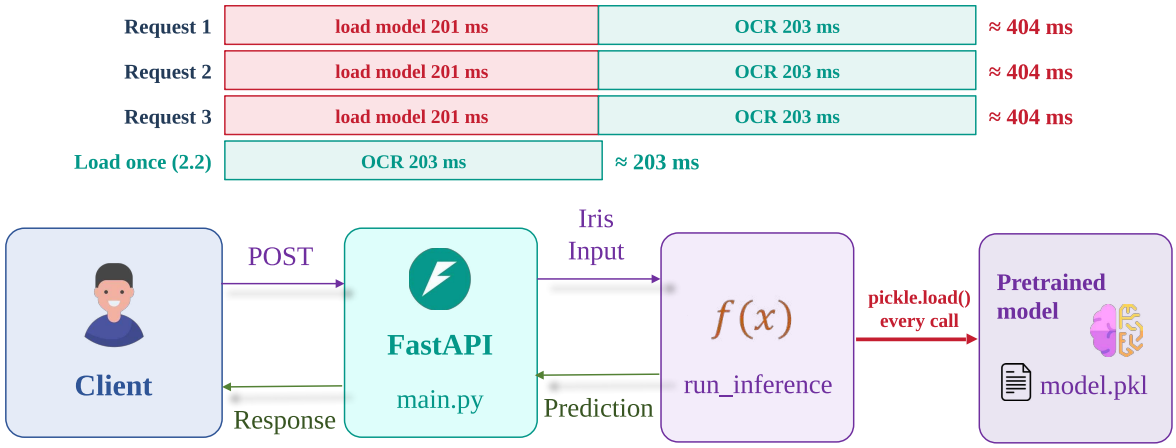
Một lỗi kinh điển khi chuyển model từ notebook sang server là **train/serve skew**, tức bước tiền xử lý ở server khác với lúc huấn luyện. Chẳng hạn, nếu server gọi `StandardScaler().fit_transform(x)` trên một mẫu duy nhất, mọi đặc trưng đều trở thành 0 vì trung bình của một mẫu chính là mẫu đó, và mọi bông hoa nhận cùng một dự đoán. Cách làm đúng là lưu toàn bộ `sklearn Pipeline`, gồm scaler đã fit và model, thành một artifact, để server chỉ việc gọi `predict`.



Hình 6: Khi server fit lại scaler trên từng request, mỗi input bị chuẩn hóa về vector 0 và hai bông hoa khác nhau nhận cùng một dự đoán; khi toàn bộ pipeline đã fit được lưu lại, server chỉ áp dụng phép biến đổi đã học và dự đoán đúng.

IV.1. Vấn đề của việc load model trong endpoint

Cách đơn giản nhất để đưa model vào API là mở `model.pkl` ngay trong hàm endpoint rồi dự đoán. Cách này cho kết quả đúng, nhưng mỗi request phải trả cả chi phí nạp model lẫn chi phí dự đoán. Với Iris model chỉ 788 byte, đo trên máy thử nghiệm cho thấy mỗi request mất khoảng 0.127 ms khi nạp lại so với 0.065 ms khi model đã nằm sẵn trong bộ nhớ (median của 200 lần). Con số này nhỏ, nhưng với model lớn hơn thì khác hẳn. Với RapidOCR, mỗi request mất khoảng 404 ms khi nạp lại (khoảng 201 ms nạp và 203 ms nhận dạng), so với khoảng 203 ms khi nạp một lần (median của 5 lần). Nói cách khác, khi lưu lượng tăng, server dành một nửa thời gian chỉ để nạp lại cùng một model, như Hình 7 cho thấy.



Hình 7: Chi phí của việc nạp model trong endpoint. Mỗi request với RapidOCR phải chờ khoảng 201 ms nạp model trước 203 ms nhận dạng, tổng cộng khoảng 404 ms, trong khi nạp một lần lúc startup chỉ còn khoảng 203 ms mỗi request; ngay cả Iris model nhỏ, khi gọi `pickle.load()` trong mỗi lần inference, cũng chậm gần gấp đôi (0.127 ms so với 0.065 ms).

IV.2. Load model một lần với lifespan và tách service layer

Giải pháp gồm hai thay đổi bổ trợ nhau. Thứ nhất, dùng **lifespan** của FastAPI để nạp model đúng một lần khi server khởi động và giải phóng khi server tắt. Thứ hai, tách logic inference ra một **service layer** để endpoint chỉ còn đảm nhận phần HTTP.

02-serve-model/main.py: lifespan, service và endpoint mỏng

```python
ml_state: dict[str, Any] = {}

@asynccontextmanager
async def lifespan(app: FastAPI):
    with open(MODEL_PATH, "rb") as f:           # STARTUP: chạy một lần
        ml_state["model"] = pickle.load(f)
    ml_state["meta"] = json.loads(META_PATH.read_text(encoding="utf-8"))
    print("[startup] model loaded once and ready.")
    yield                                       # server phục vụ request ở đây
    ml_state.clear()                            # SHUTDOWN: giải phóng
    print("[shutdown] model released.")

app = FastAPI(title="Serve a Pretrained Model", lifespan=lifespan)

def run_inference(features: list[float]) -> dict:
    model = ml_state["model"]
    # shape (1, n): chỉ là (1, 4) khi client gửi đúng 4 số
    x = np.asarray(features, dtype=float).reshape(1, -1)
    prediction = int(model.predict(x)[0])
    probabilities = model.predict_proba(x)[0]
    names = ml_state["meta"]["target_names"]
    return {"prediction": prediction, "label": names[prediction],
            "probabilities": {n: round(float(p), 4) for n, p in zip(names,
                                        probabilities)}}
```

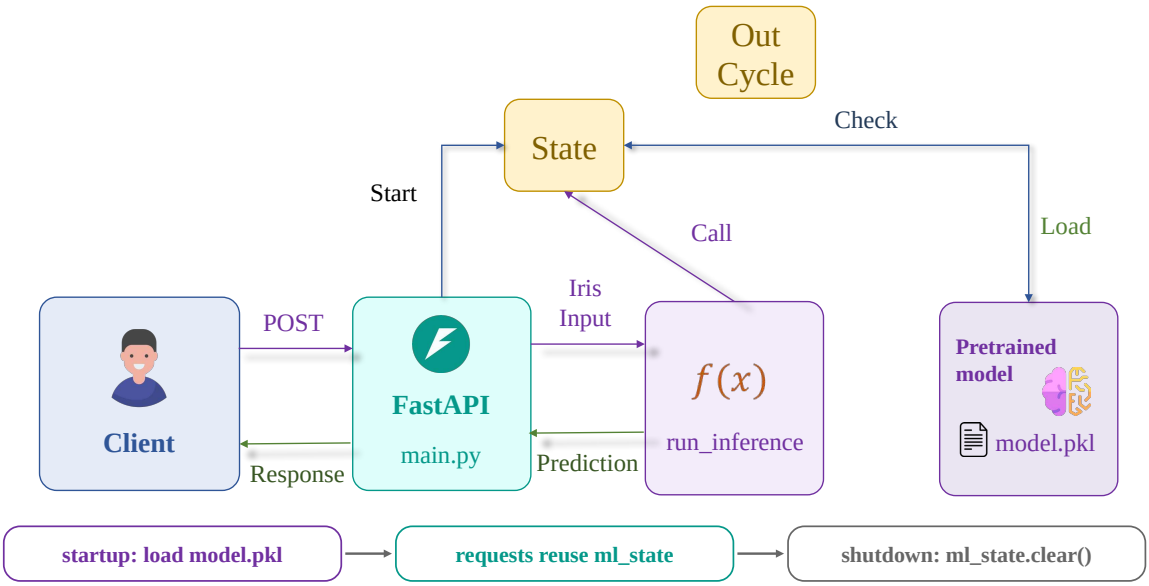
```python

@app.post("/predict")
def predict(features: list[float]):
    return run_inference(features)
```

Đọc đoạn code theo trình tự thời gian sẽ dễ hiểu hơn. Phần trước `yield` chạy khi Uvicorn khởi động; model được giữ trong `ml_state` và dùng lại cho mọi request. Phần sau `yield` chạy khi server dừng, điều đặc biệt quan trọng với model chiếm bộ nhớ GPU. Trong `run_inference`, `reshape(1, -1)` biến danh sách đầu vào thành một batch gồm một mẫu, vì sklearn luôn làm việc với batch; kích thước `(1, 4)` chỉ đạt được khi client gửi đúng bốn số, còn `reshape` không hề kiểm tra điều này; `predict` trả về chỉ số lớp còn `predict_proba` trả về xác suất của từng lớp. Endpoint `predict` giờ chỉ còn một dòng, nên khi thay model ta chỉ cần sửa service. Một lợi ích thực tế khác là nếu startup thất bại, chẳng hạn do quên chạy `train_model.py`, server sẽ không khởi động; điều này dễ phát hiện hơn nhiều so với lỗi 500 xuất hiện ở request đầu tiên. Output thực tế khi chạy server và gửi request như sau:

Output thật của 02-serve-model

```
[startup] model loaded once and ready.
POST /predict -> 200 {"prediction": 0, "label": "setosa", "probabilities": {"setosa": 0
                                          .9815, "versicolor": 0.0185, "virginica": 0
                                          .0}}
POST /predict -> 200 {"prediction": 2, "label": "virginica", "probabilities": {"setosa"
                                          : 0.0001, "versicolor": 0.0804, "virginica"
                                          : 0.9195}}
POST /predict -> 422 {"detail": [{"type": "list_type", "loc": ["body"], "msg": "Input
                                          should be a valid list", "input": {"
                                          sepal_length": 5.1}}]}
POST /predict [1, 2] -> 500 Internal Server Error   # ValueError: X has 2 features ...
POST /predict []     -> 500 Internal Server Error
[shutdown] model released.
```



Hình 8: Vòng đời của model sau khi dùng lifespan: model được nạp một lần lúc startup, mọi request `POST /predict` đi qua endpoint mỏng đến `run_inference()` và dùng chung model đã nằm sẵn trong `ml_state`, và bộ nhớ được giải phóng khi shutdown.

V. Kiểm tra dữ liệu đầu vào với Pydantic

Các dòng cuối trong output ở trên đã cho chúng ta thấy được giới hạn của đầu vào. Endpoint của bước trước nhận `list[float]` một danh sách số không tên và không có ràng buộc, và không cố định độ dài. Khi client gửi `[1, 2]` hoặc `[]`, request vẫn vượt qua bước kiểm tra data type, đến tận sklearn mới gây `ValueError` và client chỉ nhận về một lỗi 500 không giải thích gì; đây là lỗi của dữ liệu đầu vào nhưng lại hiện ra như lỗi của server. Khi client gửi `{"sepal_length": 5.1}`, họ nhận về thông báo "Input should be a valid list"khó hiểu; tệ hơn, nếu client gửi bốn số sai thứ tự hoặc số âm, model vẫn chạy và trả về một dự đoán vô nghĩa mà không ai hay biết. Server không thể kiểm soát client, vì vậy dữ liệu đầu vào phải được kiểm tra trước khi chạm tới model. Đây là vai trò của **Pydantic schema** [2], được đặt giữa endpoint và service trong thư mục `03-validate-input`:

03-validate-input/main.py: request schema và endpoint

```
class IrisFeatures(BaseModel):
    sepal_length: float = Field(..., gt=0, le=20, description="Sepal length (cm)")
    sepal_width: float = Field(..., gt=0, le=20, description="Sepal width (cm)")
    petal_length: float = Field(..., gt=0, le=20, description="Petal length (cm)")
    petal_width: float = Field(..., gt=0, le=20, description="Petal width (cm)")

    def to_vector(self) -> list[float]:
        return [self.sepal_length, self.sepal_width, self.petal_length, self.
                                                petal_width]

router = APIRouter(tags=["prediction"])

```

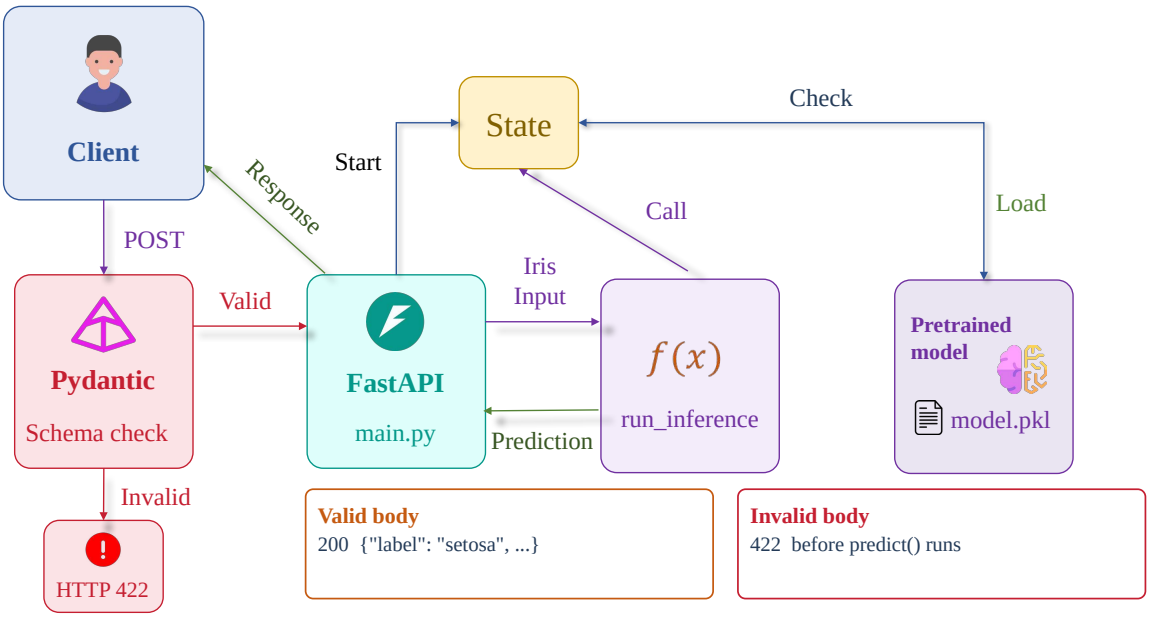
```python
@router.post("/predict")
def predict(payload: IrisFeatures):
    x = np.asarray(payload.to_vector(), dtype=float).reshape(1, -1)
    # ... predict / predict_proba như bước trước
app.include_router(router)
```

Mỗi trường trong `IrisFeatures` có tên, kiểu `float` và ràng buộc: dấu `...` biểu thị trường bắt buộc, còn `gt=0, le=20` giới hạn khoảng giá trị hợp lệ. Khi endpoint khai báo `payload: IrisFeatures`, FastAPI tự động parse JSON body, ép kiểu và kiểm tra ràng buộc; nếu body không hợp lệ, hàm `predict` sẽ không bao giờ được thực thi. Phương thức `to_vector()` giữ thứ tự đặc trưng ở một nơi duy nhất, loại bỏ nguy cơ đảo thứ tự. `APIRouter` gom các endpoint liên quan thành một nhóm để sau này dễ dàng tách ra file riêng. Trong thực tế, nên bổ sung `json_schema_extra` với một ví dụ hợp lệ để Swagger UI điền sẵn body khi thử nghiệm. Output với bốn loại body khác nhau cho thấy schema hoạt động như thế nào:

Output thật của 03-validate-input

```
POST /predict -> 200 {"prediction": 0, "label": "setosa", "probabilities": {...}}
POST /predict -> 422 {"detail": [{"type": "float_parsing", "loc": ["body", "
                                    sepal_length"], "msg": "Input should be a
                                    valid number, unable to parse string as a
                                    number", "input": "five"}]}
POST /predict -> 422 {"detail": [{"type": "greater_than", "loc": ["body", "sepal_length
                                    "], "msg": "Input should be greater than 0"
                                    , "input": -1, "ctx": {"gt": 0.0}}, {"type"
                                    : "missing", "loc": ["body", "petal_width"
                                    ], "msg": "Field required", ...}]}
POST /predict -> 422 {"detail": [{"type": "model_attributes_type", "loc": ["body"], "
                                    msg": "Input should be a valid dictionary
                                    or object to extract fields from", "input":
                                     [5.1, 3.5, 1.4, 0.2]}]}
```

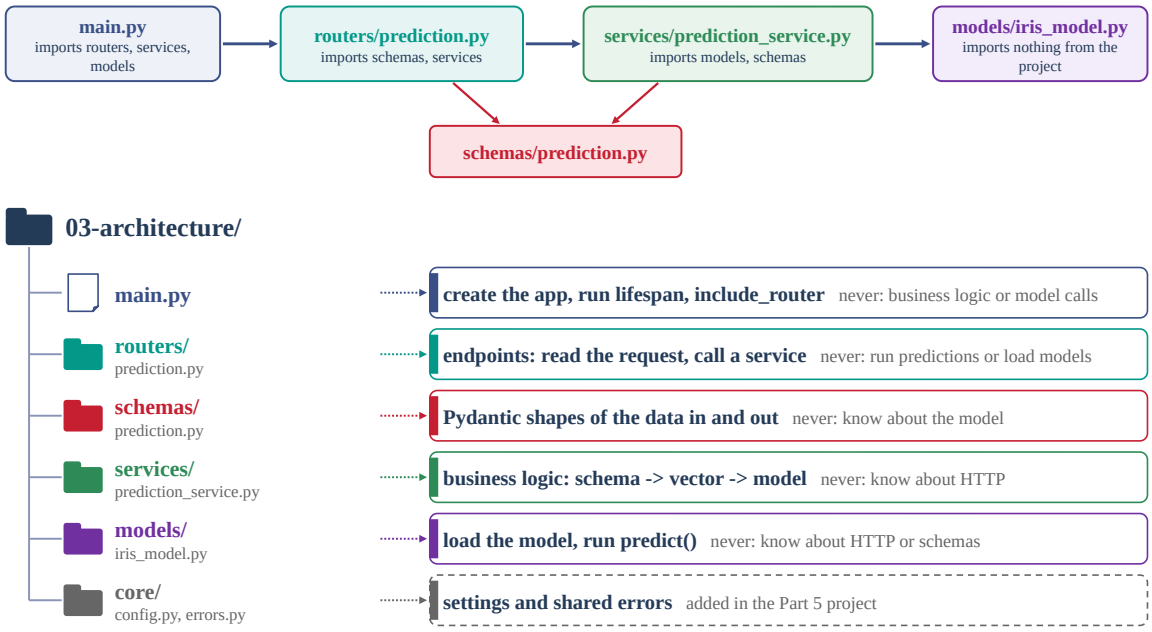
Mỗi lỗi 422 cho biết vị trí qua `loc`, loại lỗi qua `type` (`float_parsing`, `greater_than`, `missing`) và thông điệp qua `msg`. Pydantic báo tất cả lỗi trong một lần, như request thứ ba có đồng thời hai lỗi, giúp client sửa một lượt thay vì thử đi thử lại. Request cuối cho thấy body dạng list từng chạy được ở bước trước giờ bị từ chối, vì contract đã chuyển sang object có tên trường; đây cũng là lời nhắc rằng mỗi khi thay đổi schema, cần thông báo cho phía client.



Hình 9: Pydantic schema đóng vai trò cổng kiểm tra trước inference. Body hợp lệ được chuyển tới model và nhận về 200, còn body sai kiểu, thiếu trường hoặc vi phạm ràng buộc bị chặn lại với 422 trước khi model được gọi.

VI. Tổ chức source code theo layer

Sau khi bổ sung model, lifespan và validation, file `main.py` của bước trước đã dài 99 dòng và trộn lẫn mọi thứ: vòng đời model, schema, logic inference và định nghĩa route. Project cuối cùng của tài liệu có 29 file với 1,853 dòng; nếu dồn tất cả vào một file, việc đọc, kiểm thử hay làm việc nhóm gần như không khả thi. Muốn sửa bước tiền xử lý lại phải đọc cả code HTTP; logic inference trong một file vẫn có thể được kiểm thử trực tiếp, nhưng rất khó cô lập vì nó gắn chặt với biến toàn cục `ml_state` và với route. Bước này vì thế chủ yếu là một **refactor**: hành vi của `POST /predict` giữ nguyên (phiên bản này chỉ bổ sung thêm `GET /info` để xem thông tin model), nhưng code trong thư mục `03-architecture` được chia theo trách nhiệm. `main.py` chỉ tạo app, chạy lifespan và đăng ký router; `routers/` khai báo endpoint và xử lý HTTP; `schemas/` chứa các Pydantic model; `services/` chứa logic nghiệp vụ và không phụ thuộc HTTP; còn `models/` nạp model và chạy dự đoán, là layer duy nhất làm việc với sklearn. Hình 10 cho thấy cấu trúc này cùng hướng phụ thuộc giữa các layer.



Hình 10: Cấu trúc thư mục của `03-architecture`, trong đó mỗi folder đảm nhận một trách nhiệm và các lệnh import chỉ đi theo một chiều từ `main.py` qua routers và services xuống models, còn schemas được dùng chung.

Quy tắc quan trọng nhất của cấu trúc này là layer bên dưới không bao giờ import layer bên trên. Nếu `models/` import từ `routers/`, model layer sẽ bị ràng buộc vào HTTP và rất dễ phát sinh circular import. Đoạn code sau cho thấy `main.py` chỉ còn làm nhiệm vụ lắp ghép, và cách router nhận service:

03-architecture: main.py chỉ wiring, router dùng Depends

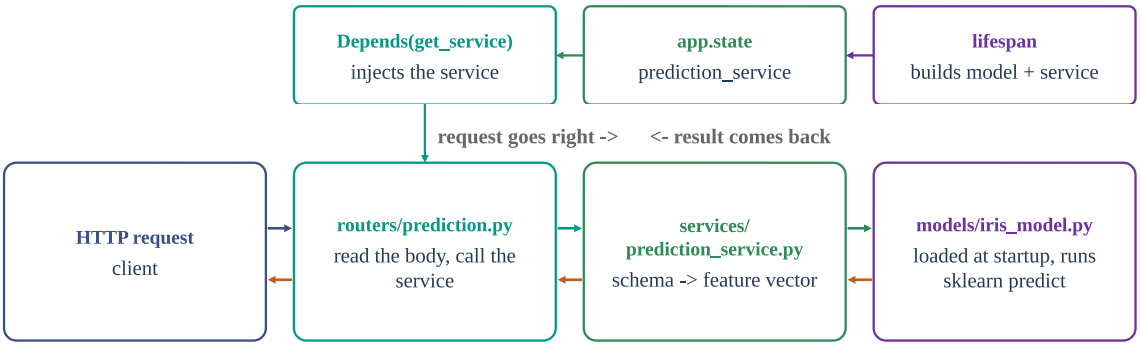
```python
# main.py
@asynccontextmanager
async def lifespan(app: FastAPI):
    iris_model = IrisModel(BASE_DIR / "model.pkl", BASE_DIR / "model_meta.json")
    iris_model.load()                                          # model layer
    app.state.prediction_service = PredictionService(iris_model)  # service layer
    yield
    iris_model.unload()

app = FastAPI(lifespan=lifespan)
app.include_router(prediction.router)                          # router layer

# routers/prediction.py
def get_service(request: Request) -> PredictionService:
    return request.app.state.prediction_service

@router.post("/predict")
def predict(payload: IrisFeatures, service: PredictionService = Depends(get_service)):
    return service.predict(payload)
```

Lifespan giờ chỉ còn lắp ghép các thành phần: tạo model, bọc nó trong service rồi lưu service vào `app.state`. Endpoint không import biến toàn cục mà khai báo `Depends(get_service)`, đây là cơ chế **dependency injection** của FastAPI. Với mỗi request, FastAPI gọi `get_service` và truyền kết quả vào tham số `service`. Lợi ích rõ ràng nhất xuất hiện khi kiểm thử: chỉ cần gán `app.dependency_overrides[get_service] = lambda: FakeService()` là có thể kiểm tra router mà không cần model thật. Bên trong, `PredictionService.predict` chỉ chuyển schema thành vector rồi gọi `IrisModel.predict`, nơi chứa đúng đoạn numpy và sklearn của bước trước. Hình 11 theo dấu một request đi qua các layer.



	Router	Service	Model
Knows HTTP?	yes	no	no
Knows sklearn?	no	no	yes

Hình 11: Hành trình của một request `POST /predict` qua các layer: router nhận service thông qua `Depends(get_service)`, service chuyển schema thành vector đặc trưng, model layer chạy sklearn, và kết quả đi ngược trở lại client.

Khi chạy lại cùng các request `POST /predict` như bước trước, kết quả giống hệt nhau (`[startup] model + service ready`, `POST /predict -> 200 {"label": "setosa", ...}`); đó chính là dấu hiệu của một lần refactor đúng. Điểm khác duy nhất về API là endpoint mới `GET /info`.

VII. Thiết kế response contract ổn định

Code đã gọn gàng, nhưng nhìn từ phía client vẫn còn một vấn đề. Response hiện là một dict tùy ý do service trả về, còn lỗi thì mỗi nơi một kiểu: lỗi 422 theo định dạng mặc định của FastAPI, lỗi 500 dưới dạng văn bản. Frontend viết code dựa trên tên các trường trong response; nếu một ngày service trả thêm trường debug, đổi tên trường, hoặc lỗi lúc là JSON lúc là văn bản, mọi client đều có thể hỏng. Điều client cần là một lời cam kết, tức một **response contract** với cấu trúc cố định cho cả trường hợp thành công lẫn thất bại.

VII.1. Dùng response_model để kiểm soát output

Bước đầu tiên là tách **request schema**, mô tả dữ liệu client gửi, khỏi **response schema**, mô tả dữ liệu server cam kết trả về, rồi gắn response schema vào endpoint. Code nằm trong thư mục `04-response-contract`:

04-response-contract: response schema và response_model

```python
# schemas/prediction.py
class ProbabilityItem(BaseModel):
    label: str
    probability: float = Field(..., ge=0, le=1)

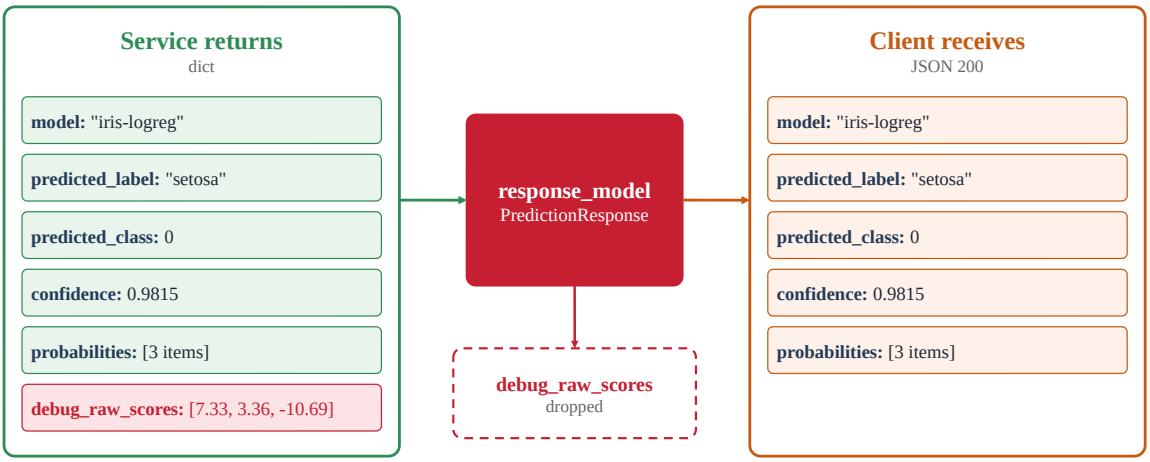
class PredictionResponse(BaseModel):
    model: str
    predicted_label: str
    predicted_class: int
    confidence: float = Field(..., ge=0, le=1)
    probabilities: list[ProbabilityItem]

# routers/prediction.py
@router.post("/predict", response_model=PredictionResponse,
             responses={400: {"model": ErrorResponse}, 422: {"model": ErrorResponse}})
def predict(payload: IrisFeatures,
            min_confidence: float = Query(0.0, ge=0, le=1),
            service: PredictionService = Depends(get_service)):
    # service trả dict có thêm `debug_raw_scores`; response_model sẽ lọc bỏ
    return service.predict(payload, min_confidence=min_confidence)
```

Tham số `response_model` làm hai việc. Nó validate output, chẳng hạn `confidence` phải nằm trong đoạn [0, 1], nên nếu service trả dữ liệu sai thì lỗi được phát hiện ngay tại server thay vì lan tới client. Đồng thời, nó lọc bỏ mọi trường không có trong schema. Tham số `responses` ghi các mã lỗi có thể xảy ra vào Swagger UI để client biết trước. Trường `probabilities` cũng được đổi từ dict sang list các `ProbabilityItem`, giúp phía client dễ duyệt và mở rộng. Output thực tế cho thấy trường nội bộ `debug_raw_scores` đã bị lọc:

Output service (RAW) và response thật sau response_model

```
RAW: {"model": "iris-logreg", "predicted_label": "setosa", ..., "debug_raw_scores": [7.
                                        332246906542891, 3.359423762120036, -10.
                                        691670668662207]}
POST /predict -> 200 {"model": "iris-logreg", "predicted_label": "setosa", "
                                        predicted_class": 0, "confidence": 0.9815,
                                        "probabilities": [{"label": "setosa", "
                                        probability": 0.9815}, {"label": "
                                        versicolor", "probability": 0.0185}, {"
                                        label": "virginica", "probability": 0.0}]}
```



Hình 12: Sự khác biệt giữa kết quả nội bộ và API response. Service có thể trả về các trường phục vụ debug như `debug_raw_scores`, nhưng `response_model` chỉ cho phép những trường đã cam kết đi tới client.

VII.2. Chuẩn hóa lỗi bằng domain exception và status code

Lỗi cũng là một phần của contract. Vì service layer không biết gì về HTTP, nó báo lỗi bằng **domain exception**; sau đó, `main.py` đăng ký các `exception_handler` để ánh xạ từng exception sang một status code và cùng một cấu trúc `{"error": {"code", "message"}}`:

04-response-contract: domain error và exception handlers

```python
# services/prediction_service.py: ngưỡng confidence là BUSINESS rule -> 400 (không phải 422)
if confidence < min_confidence:
    raise LowConfidenceError(confidence, min_confidence)

# main.py
@app.exception_handler(LowConfidenceError)
async def low_confidence_handler(request: Request, exc: LowConfidenceError):
    return JSONResponse(status_code=400,
                        content={"error": {"code": "LOW_CONFIDENCE", "message": str(exc)}})

@app.exception_handler(RequestValidationError)      # bọc 422 mặc định vào cùng contract
async def validation_handler(request: Request, exc: RequestValidationError):
    return JSONResponse(status_code=422, content={"error": {
        "code": "VALIDATION_ERROR", "message": "Request body is invalid", "details": exc.errors()}})

@app.exception_handler(Exception)                   # lỗi bất ngờ: không lộ chi tiết
async def unhandled_handler(request: Request, exc: Exception):
    return JSONResponse(status_code=500,
                        content={"error": {"code": "INTERNAL_ERROR", "message": "Internal server error"}})
# ... ModelNotFoundError -> 404 MODEL_NOT_FOUND tương tự
```

Status code được chọn theo ngữ nghĩa: 422 khi request không qua validation, dù lỗi nằm ở body, path hay query parameter; 400 khi dữ liệu hợp lệ nhưng vi phạm một quy tắc nghiệp vụ như ngưỡng confidence, đây là quy ước của demo này chứ không phải quy tắc chung; 404 khi tài nguyên được yêu cầu không tồn tại; và 500 cho lỗi bất ngờ, trong trường hợp này tuyệt đối không trả traceback cho client mà chỉ ghi log ở server. Trường `code` là một chuỗi cố định như `LOW_CONFIDENCE`, cho phép frontend rẽ nhánh xử lý mà không phải phân tích nội dung `message`. Output thực tế cho thấy các lỗi do ứng dụng xử lý đều tuân theo cùng một cấu trúc:

Output thật của 04-response-contract: lỗi do handler xử lý

```
POST /predict?min_confidence=0.99 -> 400 {"error": {"code": "LOW_CONFIDENCE", "message"
                                        : "Confidence 0.8130 is below the required
                                        threshold 0.9900"}}
POST /predict -> 422 {"error": {"code": "VALIDATION_ERROR", "message": "Request body is
                                         invalid", "details": [...]}}
GET /models/resnet50 -> 404 {"error": {"code": "MODEL_NOT_FOUND", "message": "Model '
                                        resnet50' was not found"}}
GET /debug/boom -> 500 {"error": {"code": "INTERNAL_ERROR", "message": "Internal server
                                         error"}}
```

Cần nói chính xác phạm vi của contract này. Các handler trong repo chỉ bao phủ domain exception, `RequestValidationError` (body, path hoặc query không hợp lệ, kể cả JSON sai cú pháp) và exception bất ngờ. Những lỗi do chính routing của Starlette sinh ra, như URL không tồn tại hoặc sai method, vẫn trả về định dạng mặc định của FastAPI. Ngoài ra, `ErrorResponse` trong repo chỉ khai báo `code` và `message`, trong khi response 422 thực tế có thêm `details`, và message `"Request body is invalid"` cũng được dùng cho cả lỗi query:

Output thật của 04-response-contract: lỗi nằm ngoài các handler

```
GET /nope      -> 404 {"detail": "Not Found"}
GET /predict   -> 405 {"detail": "Method Not Allowed"}
POST /predict?min_confidence=abc -> 422 {"error": {"code": "VALIDATION_ERROR", "message
                                        ": "Request body is invalid", "details": [{
                                        "loc": ["query", "min_confidence"], ...}]}}
```

Nếu muốn **mọi** response lỗi có cùng một cấu trúc, có thể bổ sung một handler cho `StarletteHTTPException` và khai báo `details` là trường tùy chọn trong error schema. Đoạn code sau là **đề xuất mở rộng, không có trong repo**:

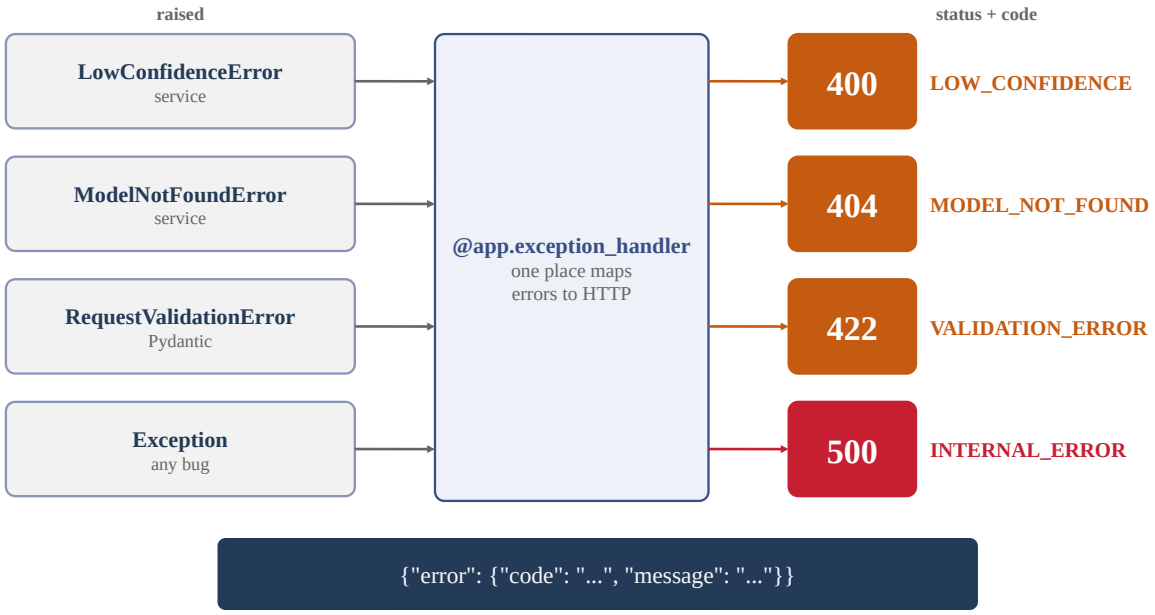
Đề xuất mở rộng (không có trong repo): error contract cho mọi lỗi

```python
from starlette.exceptions import HTTPException as StarletteHTTPException

class ErrorDetail(BaseModel):
    code: str
    message: str
    details: list[dict] | None = None      # chỉ có ở lỗi 422

```

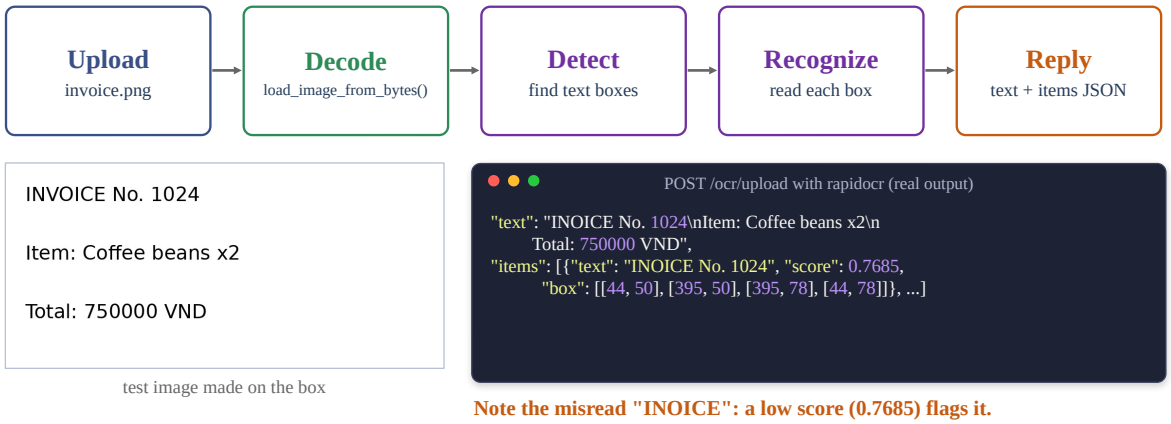
```python
# Bắt 404 (URL lạ), 405 (sai method) và HTTPException do code tự raise
@app.exception_handler(StarletteHTTPException)
async def http_error_handler(request: Request, exc: StarletteHTTPException):
    body = {"error": {"code": f"HTTP_{exc.status_code}", "message": str(exc.detail)}}
    return JSONResponse(status_code=exc.status_code, content=body,
                        headers=getattr(exc, "headers", None))
```



Hình 13: Cách mỗi domain exception được exception handler chuyển thành một status code phù hợp (400, 404, 422 hoặc 500) và một error response có cùng cấu trúc, nhờ đó client xử lý mọi lỗi theo một cách thống nhất.

VIII. Multi-model AI server cho bài toán OCR

Đến đây, server đã có đầy đủ các thành phần cơ bản, nhưng mới chỉ phục vụ một model cho một tác vụ với đầu vào JSON. Bài toán đặt ra ở phần Motivation đòi hỏi nhiều hơn thế. Người dùng cần upload ảnh hóa đơn thay vì gửi vài con số; cùng một server phải phục vụ OCR, classification (Iris) và detection (YOLOS-tiny); và có nhiều model với kích thước khác nhau, chạy trên CPU hoặc GPU. Có hai OCR model: `rapidocr` [5], dựa trên ONNX, chỉ chạy trên CPU và nặng khoảng 16 MB, và `trocr-small-printed`, một transformer khoảng 62 triệu tham số có thể chạy trên CPU hoặc CUDA. Nạp tất cả model ngay khi khởi động sẽ lãng phí RAM và VRAM cho những model có thể không bao giờ được dùng. Hình 14 minh họa luồng xử lý OCR mà server cần hỗ trợ; code nằm trong thư mục `05-multi-model-server`.



Hình 14: Luồng xử lý OCR cho một ảnh hóa đơn: ảnh được upload, decode thành ảnh RGB, các vùng chữ được phát hiện rồi nhận dạng, và server trả về văn bản cùng vị trí từng dòng; dòng bị đọc sai thành `"INOICE"` đi kèm score thấp (0.7685), cho phép client cân nhắc mức độ tin cậy.

VIII.1. Routers cho từng tác vụ và model registry

Để đáp ứng các yêu cầu này, hệ thống được bổ sung hai thành phần. Thứ nhất là **routers** theo tác vụ: mỗi tác vụ có một `APIRouter` với `prefix` riêng (`/ocr`, `/classification`, `/detection`), cùng một router `system` cho `/health`, `/devices` và `/models`; thêm một tác vụ mới chỉ cần thêm một router và một dòng `include_router`. Thứ hai là **model registry**: lifespan chỉ đăng ký cách tạo model mà không nạp ngay, model chỉ được nạp ở lần sử dụng đầu tiên (**lazy load**) và có thể được giải phóng khi cần.

05-multi-model-server/models/registry.py: lazy load và unload

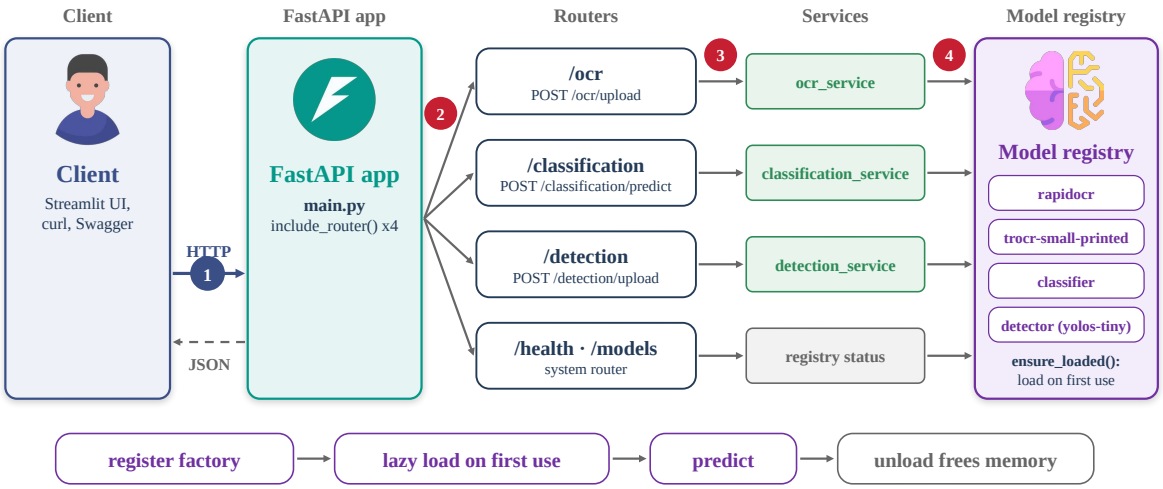
```python
class ModelRegistry:
    def __init__(self) -> None:
        self._factories: dict[str, Callable[[str], Any]] = {}
        self._instances: dict[str, Any] = {}       # "name::device" -> model instance

    def register(self, name: str, factory: Callable[[str], Any]) -> None:
        self._factories[name] = factory            # chỉ lưu cách tạo, chưa load

    def ensure_loaded(self, name: str, device: str) -> Any:
        if name not in self._factories:
            raise KeyError(name)
        key = self._key(name, device)
        if key not in self._instances:
            self._instances[key] = self._factories[name](device)
        instance = self._instances[key]
        if hasattr(instance, "is_loaded") and not instance.is_loaded:
            instance.load()                        # load ở lần dùng đầu tiên
        return instance
    # ... unload(name), unload_all(), status()
```

Registry lưu **factory**, tức hàm nhận `device` và trả về model, thay vì lưu chính model, nên server khởi động gần như tức thì dù có nhiều model nặng. Khóa `name::device` cho phép cùng một

model tồn tại đồng thời trên CPU và GPU, và nhiều router dùng chung một instance thay vì mỗi nơi nạp một bản riêng. Trong lifespan, ta gọi `registry.register(...)` cho từng model, tạo cho mỗi tác vụ một service dùng chung registry, và gọi `registry.unload_all()` khi server dừng. Cần lưu ý rằng request đầu tiên tới một model sẽ chậm hơn do phải nạp; sau khi triển khai, có thể gửi trước một request làm nóng (warm-up). Một điểm nữa: `ensure_loaded` trong repo không dùng lock. Hiện tại điều này chưa gây sự cố vì endpoint OCR chạy inference ngay trên event loop nên các request được xử lý lần lượt; nhưng khi inference được chuyển sang threadpool như đề xuất ở phần sau, nhiều request đầu tiên đến cùng lúc có thể cùng thấy model chưa được nạp và nạp nó nhiều lần, tốn thêm thời gian và bộ nhớ. Cách khắc phục là bọc phần kiểm tra và nạp model trong một `threading.Lock`. Hình 15 tổng hợp kiến trúc này.



Hình 15: Kiến trúc của multi-model server. Request từ client được router của từng tác vụ tiếp nhận, chuyển cho service tương ứng, và service lấy model từ registry, nơi model chỉ được nạp khi cần và có thể giải phóng theo device.

VIII.2. Nhận file ảnh với UploadFile

Ảnh không thể gửi gọn trong một JSON body thông thường, vì vậy endpoint OCR nhận file qua định dạng **multipart/form-data**:

05-multi-model-server/routers/ocr.py: router /ocr

```python
router = APIRouter(prefix="/ocr", tags=["ocr"])

@router.post("/upload", response_model=OCRResponse)
async def ocr_upload(
    file: UploadFile = File(..., description="Invoice or document image"),
    model: str | None = Form(None, description="OCR model name"),
    device: str | None = Form(None, description="Device to run on: cpu or cuda"),
    service: OCRService = Depends(get_service),
):
    image = load_image_from_bytes(await file.read())
    return service.extract(image, model_name=model, device=device)
```

`File(...)` và `Form(...)` báo cho FastAPI đọc các trường từ form thay vì JSON; tính năng này cần cài thêm gói `python-multipart`. Phía client gửi request bằng `requests.post(url, files={"file": open("invoice.png", "rb")}, data={"model": "rapidocr"})`. Hàm `load_image_from_bytes` trong `services/preprocess.py` kiểm tra file rỗng, giới hạn kích thước và decode ảnh, đồng thời ném `ImageDecodeError` hoặc `ImageTooLargeError` khi dữ liệu không hợp lệ. Nguyên tắc ở đây giống với Pydantic schema: dữ liệu từ client, dù là JSON hay file, luôn phải được kiểm tra trước khi đưa vào model.

VIII.3. async def và inference có tính blocking

Endpoint OCR được khai báo bằng `async def` để có thể `await file.read()`, và điều này đặt ra một câu hỏi về hiệu năng. Inference là công việc nặng về CPU và có tính blocking. Nếu một hàm `async def` gọi trực tiếp một hàm blocking, nó sẽ chặn **event loop**: với một worker, bốn request đến cùng lúc phải xử lý tuần tự, mất khoảng bốn giây nếu mỗi lần xử lý mất một giây, và trong thời gian đó cả `/health` cũng phải chờ. Endpoint OCR trong repo đang ở đúng tình huống này, vì nó gọi `service.extract(...)` trực tiếp bên trong `async def`. Có hai cách khắc phục: khai báo endpoint bằng `def`, khi đó FastAPI tự chạy nó trong **threadpool**; hoặc giữ `async def` cho phần I/O và đẩy riêng phần inference ra threadpool bằng `run_in_threadpool`. Đoạn code sau minh họa cách thứ hai, kèm lock cho registry như đã nhắc ở trên; đây là **phiên bản đề xuất, khác với repo**:

Đề xuất (khác repo): inference trong threadpool, registry có lock

```python
from fastapi.concurrency import run_in_threadpool

@router.post("/upload", response_model=OCRResponse)
async def ocr_upload(file: UploadFile = File(...), model: str | None = Form(None),
                     device: str | None = Form(None),
                     service: OCRService = Depends(get_service)):
    image = load_image_from_bytes(await file.read())   # I/O: await trên event loop
    # inference blocking: chạy trong threadpool, event loop vẫn rảnh
    return await run_in_threadpool(service.extract, image,
                                   model_name=model, device=device)

# models/registry.py: tránh nạp trùng khi nhiều thread cùng gọi lần đầu
def ensure_loaded(self, name: str, device: str) -> Any:
    with self._lock:              # tạo trong __init__: threading.Lock()
        # ... phần tạo instance và instance.load() như cũ
```

Cần phân biệt hai mục tiêu khác nhau. Threadpool giải quyết bài toán **concurrency**: event loop không bị chặn, nên server vẫn nhận request mới và trả lời `/health` trong khi inference đang chạy. Nhưng threadpool không tự động làm phép tính CPU nhanh hơn. Với code Python thuần nặng CPU, GIL chỉ cho một thread chạy bytecode tại một thời điểm; thử nghiệm trên máy 8 core cho thấy một vòng lặp mất 0.53 giây, còn bốn request chạy qua threadpool mất 2.27 giây, gần bằng chạy tuần tự. Ngược lại, với thao tác chờ như `time.sleep(1)`, bốn request chỉ mất 1.01 giây thay vì 4.01 giây. Các thư viện như ONNX Runtime hay PyTorch nhả GIL trong phần tính toán nên có thể tận dụng nhiều core, nhưng vẫn bị giới hạn bởi số core, bộ nhớ và GPU. Muốn tăng throughput thực sự cho tác vụ nặng CPU, cần thêm worker process, batching hoặc

phần cứng mạnh hơn.

VIII.4. Error có nghĩa và CORS

Khi số loại lỗi tăng lên theo số model và tác vụ, cách đăng ký một handler cho mỗi exception như ở bước trước trở nên cồng kềnh. Server tổng quát hóa error contract. Mọi lỗi kế thừa `ServiceError` và tự mang theo `status_code` cùng `code`, nên `main.py` chỉ cần một handler duy nhất:

05-multi-model-server: core/errors.py và main.py

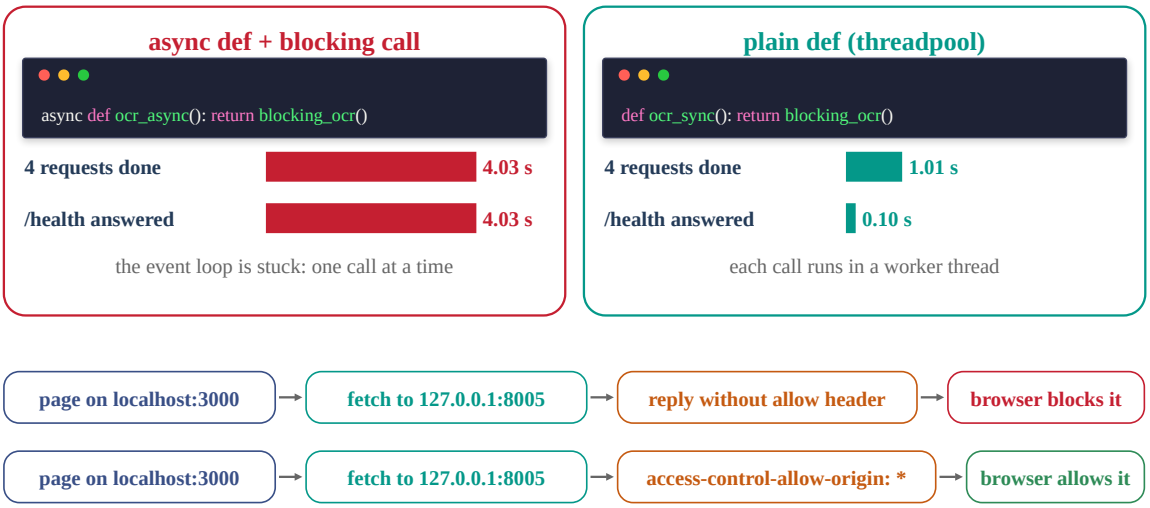
```python
class ServiceError(Exception):
    status_code = 500
    code = "INTERNAL_ERROR"

class UnknownModelError(ServiceError):
    status_code, code = 404, "UNKNOWN_MODEL"
class InvalidDeviceError(ServiceError):
    status_code, code = 400, "INVALID_DEVICE"
# ... ImageDecodeError (400), ImageTooLargeError (413), ModelBackendError (503)

app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"],
                                           allow_headers=["*"])

@app.exception_handler(ServiceError)
async def service_error_handler(request: Request, exc: ServiceError):
    return JSONResponse(status_code=exc.status_code,
                        content={"error": {"code": exc.code, "message": str(exc)}})
```

Thêm một loại lỗi mới giờ chỉ cần một subclass hai dòng mà không phải sửa handler. Đoạn code cũng đăng ký `CORSMiddleware`, liên quan đến một vấn đề xuất hiện khi giao diện web gọi API. **CORS** (Cross-Origin Resource Sharing) là cơ chế bảo vệ của trình duyệt: một trang web tại `http://localhost:3000` gọi `fetch` tới API ở một origin khác sẽ bị trình duyệt chặn nếu response không có header `Access-Control-Allow-Origin`. `CORSMiddleware` bổ sung header này. Ngược lại, Streamlit gọi API bằng `requests` từ phía server Python, không đi qua trình duyệt, nên không bị ảnh hưởng bởi CORS. Trong môi trường production, nên liệt kê cụ thể các origin được phép thay vì dùng `"*"`. Hình 16 tóm tắt cả hai vấn đề vận hành vừa nêu.



Hình 16: Hai vấn đề vận hành của server cuối cùng: khi `async def` gọi hàm blocking, bốn request mất 4.03 giây và cả `/health` cũng phải chờ, trong khi `def` chạy trong threadpool chỉ mất 1.01 giây; còn với CORS, trình duyệt chặn response thiếu header cho phép origin, và `CORSMiddleware` giải quyết điều đó.

VIII.5. Chạy toàn bộ hệ thống

Để chạy hệ thống trong một môi trường mới, từ thư mục `05-multi-model-server`, trước hết chạy `python train_model.py` để tạo artifact `model.pkl` cho Iris classifier, rồi chạy `python download_models.py` để tải trọng số TrOCR và YOLOS-tiny vào `model_cache/`; script này chỉ tải model và không tạo classifier, còn RapidOCR đã kèm sẵn trọng số trong package. Sau đó khởi động server bằng `uvicorn main:app --port 8005` và mở giao diện bằng `streamlit run app.py`. Nếu chỉ muốn thử giao diện mà chưa tải model, có thể đặt `MMS_MOCK=1`; khi đó OCR trả dữ liệu mẫu và không nạp model nào vào registry. Script `python smoke_test.py` gọi lần lượt các endpoint để kiểm tra nhanh; output thực tế (rút gọn) như sau:

Output thật của 05-multi-model-server (rút gọn)

```
[startup] device=cpu mock=False - models load lazily on demand
GET /models -> 200 {..., "models": [{"name": "rapidocr", "loaded": false, ...}, {"name"
                                    : "classifier", "loaded": false, ...}, ...]
                                    }
POST /ocr/upload -> 200 {"model": "rapidocr", "device": "cpu", "elapsed_ms": 506.83, "
                                    text": "INOICE No. 1024\nItem: Coffee beans
                                     x2\nTotal: 750000 VND", "items": [{"text":
                                     "INOICE No. 1024", "score": 0.7685, "box":
                                     [...]}, ...]}
POST /ocr/upload -> 400 {"error": {"code": "INVALID_DEVICE", "message": "Model '
                                    rapidocr' supports ['cpu'], not 'cuda'."}}
POST /ocr/upload -> 404 {"error": {"code": "UNKNOWN_MODEL", "message": "OCR model '
                                    tesseract' does not exist. Available: ['
                                    rapidocr', 'trocr-small-printed']"}}
GET /models -> 200 {..., "models": [{"name": "rapidocr", "loaded": true, "loaded_on": [
                                    "cpu"], ...}, ...]}
```

```
POST /models/rapidocr/unload -> 200 {..., "models": [{"name": "rapidocr", "loaded":
                                        false, "loaded_on": [], ...}, ...]}
[shutdown] released all models
```

Output thể hiện rõ cơ chế lazy loading. Lúc khởi động, mọi model đều ở trạng thái `"loaded": false`; sau một lần gọi OCR, `rapidocr` chuyển sang `"loaded": true` trên `cpu`; sau khi gọi `unload`, model trở lại trạng thái chưa nạp. Mọi lỗi do ứng dụng xử lý đều tuân theo cùng một contract với `code` có ý nghĩa; như đã phân tích ở phần response contract, URL không tồn tại hay sai method vẫn trả về `{"detail": ...}` mặc định nếu không bổ sung handler cho `StarletteHTTPException`. Cũng cần lưu ý kết quả `"INOICE"` bị thiếu chữ V: không model nào hoàn hảo, và đó là lý do response trả kèm `score` cho từng dòng để client tự quyết định mức độ tin cậy.

IX. Tổng kết

Tài liệu bắt đầu từ một vấn đề cơ bản: làm thế nào để một hàm Python hay một model đã huấn luyện, vốn chỉ chạy được trong notebook của người viết, trở thành một dịch vụ mà bất kỳ ứng dụng nào cũng có thể gọi tới. Câu trả lời không đến trong một bước. Hệ thống được xây dựng dần dần, và ở mỗi giai đoạn, thành phần mới chỉ được thêm vào khi phiên bản trước bộc lộ một giới hạn cụ thể. Chính cách tiếp cận tăng dần này giúp mỗi quyết định thiết kế đều có lý do rõ ràng, thay vì là một quy ước phải ghi nhớ.

Bước đầu tiên là đặt một hàm tính toán đơn giản sau một endpoint. Chỉ với một decorator và Uvicorn, hàm vốn chỉ gọi được trong cùng process đã có thể nhận request HTTP từ bất kỳ client nào, đồng thời được FastAPI tự sinh tài liệu Swagger UI. Khi thay hàm thủ công bằng một pretrained model, vấn đề hiệu năng xuất hiện ngay: nếu nạp model ở mỗi request, phần lớn thời gian phản hồi bị tiêu tốn cho việc khởi tạo thay vì inference. Lifespan giải quyết điều này bằng cách nạp model đúng một lần khi server khởi động, còn service layer tách logic inference khỏi phần xử lý HTTP để endpoint trở nên mỏng và dễ kiểm thử.

Tiếp theo, hệ thống phải đối mặt với thực tế rằng dữ liệu từ client không bao giờ đáng tin tuyệt đối. Pydantic schema đóng vai trò lớp phòng vệ đầu tiên: request sai kiểu hoặc thiếu trường bị từ chối với lỗi `422` trước khi chạm tới model. Khi số lượng code tăng lên, một file `main.py` duy nhất không còn phù hợp; việc tổ chức source code theo layer cùng cơ chế `Depends()` giúp mỗi thành phần có một trách nhiệm rõ ràng và có thể thay thế độc lập. Sau đó, response contract với `response_model` và exception handler bảo đảm rằng client nhận được cùng một cấu trúc dữ liệu cho response thành công và cho các lỗi do ứng dụng xử lý, nhờ vậy phía frontend có thể xử lý mọi tình huống một cách nhất quán. Cuối cùng, toàn bộ các nguyên tắc trên được kết hợp trong một multi-model server cho bài toán OCR, nơi routers phân chia API theo nhóm chức năng, model registry quản lý vòng đời của nhiều model trên các device khác nhau, `UploadFile` tiếp nhận ảnh, và CORS cho phép ứng dụng web gọi tới server.

Giai đoạn	Giới hạn cần khắc phục	Thành phần bổ sung
Đưa hàm Python lên API	Hàm chỉ gọi được trong cùng process	Endpoint và Uvicorn
Serve pretrained model	Nạp model ở mỗi request tốn chi phí	Lifespan, service layer
Kiểm tra đầu vào	Dữ liệu từ client không đáng tin	Pydantic schema, lỗi 422
Tổ chức source code	Một file đảm nhận mọi trách nhiệm	Cấu trúc theo layer, `Depends`
Response contract	Response và lỗi không ổn định	`response_model`, exception handler
Multi-model server	Nhiều tác vụ, model và device	Routers, model registry, UploadFile, CORS

Nhìn lại quá trình này, chúng ta có thể rút ra một số nguyên tắc chung. Thứ nhất, những tài nguyên đắt đỏ như model cần được quản lý theo vòng đời của server chứ không theo vòng đời của request. Thứ hai, ranh giới giữa các layer, gồm HTTP, validation, nghiệp vụ và model, nên được giữ rõ ràng ngay từ sớm, vì chi phí tách chúng ra về sau luôn lớn hơn nhiều. Thứ ba, một API tốt được đánh giá qua sự ổn định của contract mà nó cam kết với client, chứ không chỉ qua độ chính xác của model phía sau. Và cuối cùng, mọi model đều có thể sai, nên response nên mang theo những thông tin như `score` để client tự đánh giá mức độ tin cậy.

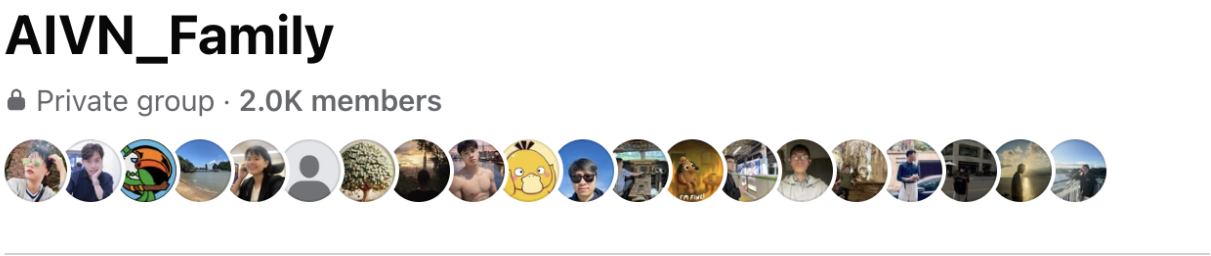
Hệ thống hiện tại đã đủ vững để phục vụ model trong môi trường phát triển, nhưng con đường đến production vẫn còn tiếp diễn. Bước tự nhiên kế tiếp là đóng gói server bằng Docker để chạy nhất quán trên mọi môi trường, sau đó deployment lên hạ tầng thật với domain và HTTPS. Khi dịch vụ được công khai, authentication bằng API key hoặc token cùng rate limiting trở nên cần thiết để bảo vệ tài nguyên. Ở góc độ hiệu năng, batching nhiều request vào một lần inference giúp tận dụng GPU tốt hơn, còn automated testing cho từng layer giúp hệ thống có thể thay đổi mà không sợ phá vỡ contract. Với các bài toán quy mô lớn hơn, những serving framework chuyên dụng như Triton Inference Server, TorchServe hay vLLM cung cấp sẵn nhiều tối ưu này. Dù vậy, các nguyên tắc đã xây dựng trong tài liệu, từ quản lý vòng đời model, validation, phân tầng đến response contract, vẫn là nền tảng để hiểu và sử dụng hiệu quả những công cụ đó.

Phụ lục

1. **Solution:** Các file code cài đặt hoàn chỉnh và phần trả lời nội dung trắc nghiệm có thể được tải tại đây.

2. **Video demo:** Quét mã QR bên dưới hoặc nhấn vào đây để xem video demo ứng dụng OCR được xây dựng trong bài học.

3. **Q&A:** Mọi người có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại đây. Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 tiếng.



References

[1] FastAPI Documentation, *FastAPI*, https://fastapi.tiangolo.com/, Accessed 17 October 2026.

[2] Pydantic Documentation, *Pydantic*, https://docs.pydantic.dev/latest/, Accessed 17 October 2026.

[3] Uvicorn Documentation, *Uvicorn: An ASGI web server for Python*, https://github.com/encode/uvicorn, Accessed 17 October 2026.

[4] Nguyen Dang Nha, *routes-for-AI-inference: Source code for Building Routes for AI Inference using FastAPI*, https://github.com/dangnha/routes-for-AI-inference, 2026.

[5] RapidAI, *RapidOCR*, https://github.com/RapidAI/RapidOCR, Accessed 17 October 2026.